

Pandas

Tabular Data Processing Review Notes

This chapter upgrades NumPy's one- and two-dimensional arrays into a table with named columns, row indexes, and mixed data types: a `DataFrame`. The goal is not to memorize methods. First ask what shape the real data has: independent records or a continuous signal, grouping by category or comparing across dimensions? That decision points directly to the right function.

- Python 3 - Pandas
- tutor style
- visual explanations
- FAQ included
- 12 core operation groups

00

Decision Table and Review Order

First identify the shape of the data and the operation you need. The answer usually tells you which Pandas tool to use.

What you need to do	Recommended tool
Create a table	<code>pd.DataFrame(dict)</code>
Filter rows by a condition	<code>mask = df["col"] > x; df[mask]</code>
Calculate one statistic per category	<code>df.groupby("col")["other"].mean()</code>
Count values in a column	<code>df["col"].value_counts()</code>
Add a column without changing the original	<code>df.copy()</code> , then assign
Handle missing values in independent records	<code>fillna(median/mean)</code>
Handle gaps in a continuous signal	<code>interpolate()</code> / <code>ffill()</code> / <code>bfill()</code>
Express a rule with no built-in vectorized method	<code>apply(func, axis=1)</code>
Read or write tabular files	<code>pd.read_csv</code> / <code>to_csv</code>
Work with time data	<code>pd.to_datetime</code> + <code>.dt</code>
Join two tables horizontally by a shared key	<code>pd.merge(..., on="id")</code>
Stack tables vertically	<code>pd.concat(..., ignore_index=True)</code>
Inspect variables and relationships	<code>df.describe()</code> / <code>df.corr()</code>
Smooth a signal or inspect window variation	<code>series.rolling(window).mean()</code> / <code>.std()</code>
Turn a long table into a comparison grid	<code>df.pivot_table(...)</code>

01 - START DataFrame structure	02 Boolean mask filter rows	03 groupby aggregate groups	04 .copy() add columns	05 Missing values two strategies	06 CSV I/O + inspection
07 apply custom rules	08 datetime time handling	09 corr relationships	10 merge/concat combine tables	11 rolling windows	12 pivot_table comparison grid

Connection to NumPy: think of a DataFrame as "a NumPy array + column names + a row index." Boolean masking and vectorized operations transfer directly, but the object changes from a numerical array to structured tabular data. `groupby` , time handling, and table joins are the main new tools.

01 Creating and Inspecting a DataFrame

• Core concept

```
dict: {"patient_id": [101, 102, 103], "heart_rate": [72, 105, 88]}
```

patient_id	heart_rate
101	72
102	105

key -> column name
same position -> one row

Every dictionary key becomes a column; values at the same position form a row.

Create a DataFrame and inspect basic properties

dataframe

```
patient_data = {
    "patient_id": [101, 102, 103],
    "heart_rate": [72, 105, 88],
}
df = pd.DataFrame(patient_data)

df.shape    # (3, 2) -> (rows, columns)
df.dtypes   # The type inferred by pandas for each column
```

Difference from a NumPy array: a NumPy array requires all elements to share one type and has no column names, only positional indexes. Every DataFrame column can have its own name and type - numbers, strings, or datetimes. It still supports vectorized computation, while also allowing dictionary-like access by column name.

02 Boolean Masks: Filtering DataFrame Rows

• Core concept

As in NumPy, a mask is still used for filtering. The object being indexed changes from an array to an entire table.

Filter rows with Boolean conditions

mask

```
high_heart_rate_mask = df["heart_rate"] > 100
low_oxygen_mask = df["oxygen_level"] < 95
combined_mask = high_heart_rate_mask | low_oxygen_mask

df[combined_mask]    # Complete rows that satisfy a condition, not one column
```

Use case: any question that asks for patients satisfying conditions follows the same Boolean logic as NumPy. The only change is that a one-dimensional array becomes a whole table: filtering returns complete rows, not a single numeric value.

03 groupby: Aggregate by Category

• Core concept

```
df.groupby("department")["heart_rate"].mean()
```

cardiology

neurology

-> one Series

split -> select a column -> aggregate

```
df.groupby("department")["heart_rate"].mean()
# Read it as: group by department
#           examine only the heart_rate column in each group
#           calculate mean() for every group
# Result: a Series whose index is the category and value is its mean
```

• Code template

```
def mean_by_group(dataframe, group_column, value_column):
    return dataframe.groupby(group_column)[value_column].mean()
```

Use case: questions such as "average, sum, or count by department, date, or diagnosis label" are among the most common tabular-data tasks.

04 Add a Calculated Column Safely with .copy()

• Core concept

```
def add_fever_column(dataframe, threshold=38.0):
    data_copy = dataframe.copy()
    data_copy["has_fever"] = data_copy["temperature"] >= threshold
    return data_copy
```

The same lesson as NumPy view/copy: adding a column directly to the supplied **dataframe** also changes the caller's original table. Copy first, add the column to the copy, and return it. This makes the function safer and easier to test.

• Code template

```
def add_calculated_column(dataframe, threshold):
    copied = dataframe.copy()
    copied["new_column"] = copied["some_column"] >= threshold
    return copied
```

• Common mistake

Incorrect	Why it fails	Correct
Add a column directly to the supplied dataframe	It also changes the caller's original table, creating a hidden side effect.	Use <code>copied = dataframe.copy()</code> , then add the column to <code>copied</code> .

05 Missing Values: fillna for Tables, interpolate/ffill/bfill for Signals

• Core concept

signal = [70, nan, nan, 76]

interpolate -> 72.0, 74.0 (linear transition)

reasonable for a continuous signal

ffill -> 70.0, 70.0 (copy the previous value)

assumes the sensor was stuck

The same gap can produce different values under different assumptions.

```
# Independent rows: statistical imputation is usually reasonable
median = df["oxygen_level"].median()
df["oxygen_level"] = df["oxygen_level"].fillna(median)

# One sensor over time: before/after values are more informative
signal.interpolate() # Linear transition when change is gradual
signal.ffill()       # Copy the previous valid value
signal.bfill()       # Copy the next valid value; useful for a leading gap
```

Decision rule: independent rows from different patients -> `fillna(median/mean)`. Repeated measurements along one time sequence -> `interpolate/ffill/bfill`. The essential question is whether the records are unrelated or form one continuous signal.

Common mistakes

Incorrect	Why it fails	Correct
Use <code>ffill</code> on unrelated patient rows	The previous patient's value has no clinical meaning for the next patient.	<code>fillna(median)</code>
Fill a continuous signal with one fixed median	It erases the real trend of the signal.	<code>interpolate / ffill / bfill</code>

06 CSV I/O and describe as a First Check

Core concept

Read -> inspect -> export

CSV

```
df = pd.read_csv(path) # Read a file; pandas infers column types
df.describe()         # count/mean/std/min/25%/50%/75%/max for numeric columns
filtered.to_csv(output_path, index=False)
# index=False avoids writing a meaningless extra index column
```

`describe()` **count is the first missing-value check:** if a column's count is smaller than the total row count, that column contains missing values. Run this once before most analyses until it becomes automatic.

Common mistake

Risky approach	Why it is risky	Correct approach
Immediately apply or iterate over a column containing NaN	<code>.mean()</code> skips NaN automatically, but row-wise rules may quietly propagate it.	Run <code>df.describe()</code> first and check whether count equals the row total.

07 apply(): Custom Rules When No Vectorized Method Fits

Core concept

`axis=1` passes one row to the function

apply

```
def compute_deviation_score(row):
    deviation = 0
    if row["heart_rate"] > 100:
        deviation += row["heart_rate"] - 100
    return deviation

df["risk_score"] = df.apply(compute_deviation_score, axis=1)

# Series.apply does not need axis because the input is already one column
df["risk_category"] = df["risk_score"].apply(categorize_score)
```

Meaning of axis=1: `apply` hands your function one complete row - a Series whose values can be read with `row["column"]` - and collects the returned values into a new column. If you omit `axis=1`, the default `axis=0` passes whole columns instead, so the dimensions do not match the intended rule.

• Code template

Template 4 - custom row rule

apply

```
def score_row(row):
    score = 0
    if row["heart_rate"] > 100:
        score += row["heart_rate"] - 100
    return score

def add_score_column(dataframe):
    copied = dataframe.copy()
    copied["score"] = copied.apply(score_row, axis=1)
    return copied
```

• Common mistake

Incorrect	Why it fails	Correct
<code>dataframe.apply(compute_deviation_score)</code>	The default axis=0 passes whole columns as Series, not rows.	<code>dataframe.apply(func, axis=1)</code>

08 datetime: Convert, Sort, Group by Day, and Filter a Time Range

• Core concept

Convert first, then use .dt and time comparisons

datetime

```
df["timestamp"] = pd.to_datetime(df["timestamp"])
# Convert strings into real datetime objects
df = df.sort_values("timestamp").reset_index(drop=True)

df.groupby(df["timestamp"].dt.date)["heart_rate"].mean()
df["timestamp"].dt.day_name()

start_time = pd.Timestamp("2026-07-01 09:00")
end_time = pd.Timestamp("2026-07-02 09:00")
mask = (df["timestamp"] >= start_time) & (df["timestamp"] <= end_time)
df[mask]
```

The **.dt accessor**: it extracts parts such as date, weekday, or hour from a datetime column. The column must first be converted with **pd.to_datetime**; do not use **.dt** directly on a string column.

• Code template

Template 5 - prepare a time series

timeseries

```
def prepare_timeseries(dataframe, timestamp_col="timestamp"):
    dataframe = dataframe.copy()
    dataframe[timestamp_col] = pd.to_datetime(dataframe[timestamp_col])
    return dataframe.sort_values(timestamp_col).reset_index(drop=True)
```

• Common mistake

Incorrect	Why it fails	Correct
<code>df["timestamp"] > "2026-07-01"</code> while the column is still text	.dt and reliable time comparisons require a real datetime type; a date-looking string is not enough.	Convert first with <code>pd.to_datetime(...)</code> .

09 describe + corr: Statistical Overview and Correlation

• Core concept

Correlation matrix and the strongest related column

corr

```
df[numeric_columns].describe() # Mean, standard deviation, and quantiles
df[numeric_columns].corr()     # Pairwise correlation matrix

# Find the other column most strongly related to one column
corr = corr_matrix[column].drop(column) # Remove self-correlation 1.0
best = corr.abs().idxmax()              # Largest absolute relationship
value = corr[best]                     # Recover the signed correlation
```

A correlation matrix always has 1.0 on its diagonal because every variable is perfectly correlated with itself. This is normal, not a bug. After receiving a new dataset, first inspect whether any variables are strongly related - perhaps redundant, perhaps informative - and only then decide how to model them.

• Code template

Template 6 - find the most correlated column

correlation

```
def most_correlated_with(corr_matrix, column):
    corr = corr_matrix[column].drop(column)
    best = corr.abs().idxmax()
    return best, corr[best]
```

• Common mistake

Incorrect

Why it fails

Correct

```
corr_matrix["heart_rate"].idxmax()
```

The diagonal value is always 1.0, so without excluding it the result is always the same column.

```
corr.drop(col).abs().idxmax()
```

10 merge / concat / value_counts / sort_values

• Core concept



merge combines columns by a key; **concat** stacks similarly structured tables.

Four essential combining and organizing operations

merge/concat

```
pd.merge(demographics, vitals, on="patient_id")
# Align rows by patient_id even when the two tables use different row orders

df["department"].value_counts()
# Like Counter, but returns a Pandas Series

df.sort_values("heart_rate", ascending=False).reset_index(drop=True)
# Sort, then replace the old row labels with 0, 1, 2, ...

pd.concat([day1, day2], ignore_index=True)
# Stack tables and create a fresh continuous row index
```

One-sentence distinction: **merge** is a horizontal join - different information from different tables becomes one row. **concat** is a vertical stack - tables with the same structure, such as measurements from two days, are placed one after another.

• Code template

Template 7 - combine two tables

merge

```
merged = pd.merge(table_a, table_b, on="patient_id")
combined = pd.concat([day1, day2], ignore_index=True)
```

11 rolling: Smooth a Signal and Inspect Variation

• Core concept

window=3: each result examines itself and the previous two points

NaN	NaN	avg	avg
-----	-----	-----	-----

the first window-1 positions are NaN

rolling(3).mean() smooths sudden high and low noise

Rolling mean and standard deviation

rolling

```
signal.rolling(window=3).mean()
# The average of each point and the previous two points; smooths noise
signal.rolling(window=3).std()
# Variation within the same window; larger values mean instability
```

The first window-1 positions are NaN, and that is expected: there are not yet enough observations to form a full window. Use cases include denoising continuous sensor data and checking whether the recent period is becoming unstable. It is smoother than **diff**, which only compares adjacent points.

12 pivot_table: Turn a Long Table into a Comparison Grid

• Core concept

Long table: department, day, heart_rate - one record per row

->

index = department
columns = day
values = heart_rate aggfunc = mean

A long table becomes a department-by-day grid for direct comparison.

The four pivot_table arguments

pivot

```
df.pivot_table(
    values="heart_rate", index="department",
    columns="day", aggfunc="mean"
)

# values -> numeric column to aggregate
# index -> labels for result rows
# columns -> labels for result columns
# aggfunc -> how each grid cell is aggregated: mean/sum/count...
```

Use case: when you need to compare a metric across two dimensions at once - for example department and date. **groupby** usually groups by one dimension and returns a Series; **pivot_table** cross-groups by two dimensions and returns a row-by-column grid that can be compared directly or plotted as a heatmap.

Common Error Summary

Incorrect	Why it fails	Correct
Add a column directly to the supplied dataframe	It changes the caller's original table.	Use <code>.copy()</code> first.
Use a NaN column without checking <code>describe()</code>	NaN may quietly spread through apply or row-wise rules.	Run <code>describe()</code> first and inspect count.
Use <code>ffill</code> on unrelated patient rows	The previous patient's value has no meaning for the next patient.	<code>fillna(median)</code>
<code>corr_matrix[col].idxmax()</code>	The self-correlation diagonal is 1.0.	Drop col, then use <code>abs().idxmax()</code> .
<code>df.apply(func)</code> and forget <code>axis=1</code>	The default passes columns, not rows.	<code>df.apply(func, axis=1)</code>
Use <code>.dt</code> directly on a string column	<code>.dt</code> requires a real datetime dtype.	Convert with <code>pd.to_datetime(...)</code> .

FAQ

Q. What is the fundamental difference between a DataFrame and a NumPy array?

A NumPy array requires all elements to share one type and has no column names, only positional indexes. A DataFrame is a table: each column can have its own name and type - numbers, strings, and times can coexist. It supports NumPy-like vectorized computation and dictionary-like access by column name. Think of it as "a NumPy array + column names + a row index."

Q. What is the difference between `df["col"]` and `df[["col"]]`?

`df["col"]` returns a one-dimensional Series. `df[["col"]]` returns a one-column DataFrame because the selector is a list. The latter is useful when downstream code must keep a table-shaped, two-dimensional structure.

Q. Are Counter and value_counts the same thing?

They are conceptually similar, but `Counter` is a general Python tool used on lists, while `value_counts()` is a Pandas method used on a DataFrame column - a Series. Its result is also a Series and can be sorted or plotted directly.

Q. How do I choose between merge and concat?

If two tables share a key column and contain complementary information - for example demographics and laboratory results - use `merge`. If the tables have exactly the same structure but represent different batches or days, use `concat`. One is a horizontal join; the other is a vertical stack.

Q. What does `reset_index(drop=True)` do?

After sorting or filtering, a DataFrame keeps its original row labels, so the rows may now display labels such as 1, 3, and 5. `reset_index(drop=True)` replaces them with 0, 1, 2, Without `drop=True`, the old index is preserved as an extra column.

Q. What is the difference between rolling and diff?

`diff` only measures the difference between adjacent points and makes the result look sharp. `rolling(window).mean()` averages over a window and is better for smoothing noise; `rolling(window).std()` measures variation within the window and helps detect instability. `diff` shows an instant change; `rolling` shows the state of a recent period.

Q. Are pivot_table and groupby the same thing?

Both perform grouped aggregation. `groupby` usually groups by one dimension and returns a Series or a simple table. `pivot_table` cross-groups by two dimensions - one for rows and one for columns - and returns a grid suited to direct comparison or heatmaps.

15 Exercises

• Practice problems

- E Filter and add a column:** `patient_id=[1,2,3,4]`, `heart_rate=[72,130,88,105]`. Add `is_urgent` for heart rate above 100, return urgent IDs, and sort by heart rate descending. Expected IDs: `[2,4]`. Practice `.copy()`, Boolean masks, and `sort_values`.
- E Find the maximum by department:** departments `[A,B,A,B]`, heart rates `[72,105,90,98]`. Return the highest rate per department. Expected: A=90, B=105. Practice `groupby` plus `max()`.
- M Compare missing-value strategies:** for `[70,nan,nan,76]`, calculate both `interpolate()` and `ffill()` and explain the difference. Expected interpolation: 72.0 and 74.0; forward fill: 70.0 and 70.0.
- M Create a custom label with apply:** scores `[0,3,8]`; `0 -> "ok"`, `0 < score <= 5 -> "watch"`, `score > 5 -> "urgent"`. Expected: `["ok", "watch", "urgent"]`. Practice `Series.apply()`.
- M Merge and count:** join `info(patient_id, dept)` with `vitals(patient_id, heart_rate)`, then count patients per department. Expected A=2, B=1. Practice `merge(on="patient_id")` plus `value_counts()`.

16 Review Order and Chapter Checklist

• Must master

Create with `pd.DataFrame(dict)`; understand shape and dtypes

Boolean filtering: `df[df["col"] > x]`

`groupby(...)[...].mean()` for category aggregation

Use `.copy()` before adding a column

Choose `fillna` vs `interpolate/ffill/bfill` by data structure

`apply(func, axis=1)` for custom row rules

• Recommended practice

`pd.read_csv` / `to_csv(index=False)`

`pd.to_datetime` + `.dt` accessor

`describe()` / `corr()` for a first dataset inspection

merge horizontally vs concat vertically

`value_counts` / `sort_values` / `reset_index(drop=True)`

`rolling(window).mean()` / `.std()`

`pivot_table` for a two-dimensional comparison grid

• Next steps

MultiIndex

merge `how=left/right/outer`

resample time series

Pandas + matplotlib plots

performance: precise dtypes and categorical data

move into ML feature engineering

• Chapter checklist

- A DataFrame is a named-column, mixed-type table rather than a pure numerical array.
- `groupby(...)[...].aggregation()` is the standard category-summary pattern.
- Use `fillna` for independent records and interpolation/ffill/bfill for continuous signals.
- Convert time columns with `pd.to_datetime` before using `.dt`.
- `merge` joins by a shared key; `concat` stacks tables with the same structure.
- Boolean masking works like NumPy but filters complete rows.
- Create a copy before adding or changing a column.
- `apply(func, axis=1)` runs a function once per row; `axis=0` is the default.
- Drop self-correlation before using `abs().idxmax()`.
- `rolling` supports smoothing and variation; `pivot_table` builds a two-dimensional comparison.

After every exercise: you can now work with real-world datasets stored in files, indexed by time, and containing missing values. Data cleaning and feature engineering are the most common steps before scikit-learn modeling. The next step is to hand the cleaned DataFrame directly to a scikit-learn model.